

# Levenshtein distance zadatak

David Katrinka 26123035

## 1. Opis zadatka

Cilj zadatka je izrada web aplikacije koja pronalazi najbližnju reč iz baze podataka u odnosu na reč koju korisnik unese. Sličnost se određuje Levenshtein rastojanjem (broj operacija zamene, brisanja ili dodavanja karaktera potrebnih da se jedna reč pretvori u drugu).

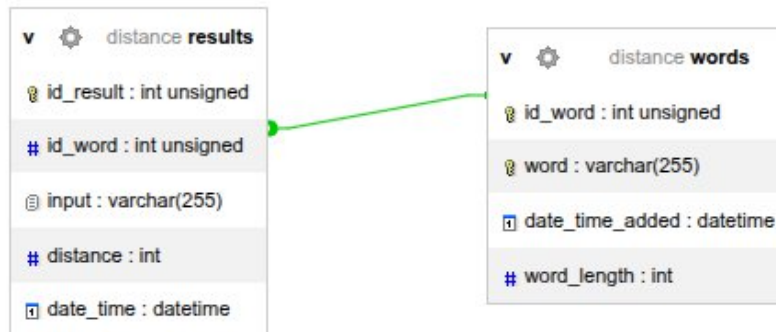
### Funkcionalnosti aplikacije:

- Korisnik unosi reč u tekstualno polje na početnoj stranici.
- Biraju se minimalna i maksimalna dužina reči za pretragu (3–15 karaktera).
- Na osnovu unete reči i baze podataka pronalazi se najbližnja reč.
- Rezultat se prikazuje u tabeli: uneta reč, najbližnja reč, vrednost Levenshtein rastojanja.
- Svaki unos i rezultat se čuva u tabeli results.

## 2. Korišćene tehnologije

- PHP 8+ – serverski jezik
- MySQL / MariaDB – baza podataka
- PDO – za rad sa bazom podataka
- Composer – za upravljanje bibliotekama
- Bootstrap 5 – stilizovanje frontenda

### 3. Struktura baze podataka



### 4. Ključni PHP kod

#### 4.1 Funkcija za dobavljanje reči po dužini

```
function getWordsInRange(int $minLength = 3, int $maxLength = 15): array
{
    $pdo = $GLOBALS['pdo'];
    $sql = "SELECT id_word, word FROM words WHERE word_length BETWEEN :min
AND :max";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([':min' => $minLength, ':max' => $maxLength]);

    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}
```

#### 4.2 Funkcija za Levenshtein

```
function findClosestWord(string $inputWord, array $words): ?array
{

```

```

$closestWord = null;

$smallestDistance = null;

foreach ($words as $row) {

    $distance = levenshtein($inputWord, $row['word']);

    if ($smallestDistance === null || $distance < $smallestDistance) {

        $smallestDistance = $distance;

        $closestWord = $row;

    }

}

if ($closestWord === null)

    return null;

$closestWord['distance'] = $smallestDistance;

return $closestWord;

}

```

## 4.3 Funkcija za unos u results

```

function insertResult(int $idWord, string $inputWord, int $distance): void
{

    $pdo = $GLOBALS['pdo'];

    $sql = "INSERT INTO results (id_word, input, distance, date_time)

        VALUES (:id_word, :input, :distance, NOW())";

```

```

$stmt = $pdo->prepare($sql);

try {
    $stmt->execute([
        ':id_word' => $idWord,
        ':input' => $inputWord,
        ':distance' => $distance
    ]);
} catch (PDOException $e) {
    if ($e->getCode() == 23000) {
    } else {
        throw $e;
    }
}
}

```

## 5. Optimizacija i performanse

- Indeks idx\_word\_length smanjuje broj kandidata nad kojima se izračunava Levenshtein rastojanje.
- Filtriranje po dužini (WHERE word\_length BETWEEN :min AND :max) smanjuje dataset i ubrzava proces.
- PHP funkcija levenshtein() je dovoljno brza za reči dužine 3–15 karaktera, čak i sa 50.000+ reči.

## 6. Frontend

- Bootstrap 5 table sa card, table-striped, table-hover, i badge za distance
- Minimalni i maksimalni select inputi za dužinu reči (3–15)
- Jednostavna i pregledna forma sa validacijom

## 7. Zaključak i predlog za unapređenje

- Aplikacija uspešno pronalazi najslićnije reči pomoću Levenshtein distance.
- Baza i indeksiranje omogućavaju rad sa velikim brojem reči bez značajnog usporavanja.
- Moguća poboljšanja:
  - Dodati AJAX pretragu za trenutno prikazivanje rezultata bez reload-a stranice.
  - Implementirati cache ili predračunavanje distance za najčešće unesene reči.
  - Omogućiti korisnicima unos više reči odjednom.